MIESC: A Multi-layer Framework for Automated Smart Contract Security Evaluation

***Abstract*—**Automated smart contract security tools tend to specialize: static analyzers cover known code patterns, fuzzers exercise reachable states, and LLM-based auditors can reason about higher-level intent but vary across runs. This paper studies whether a reproducible orchestration of these techniques improves coverage for smart-contract triage. MIESC combines 35 analysis modules across 9 layers, normalizes their outputs, applies RAG-backed false-positive filtering, and records the artifacts needed to reproduce each reported claim. On SmartBugs-curated (143 contracts), the primary static+intelligence profile reports 95.8% recall (137/143); a secondary local 32B follow-up over the remaining misses raises this to 97.9% (140/143). On 11 real DeFi exploits ($1.59B in evaluated losses), MIESC detects 9/11 cases (81.8%; Cohen's =0.773 on this small sample). On an EVMBench local high-severity extraction (40 audits, 120 findings), a four-provider union ensemble detects 111/120 vulnerabilities (92.5% recall), compared with 82.5% for the best single provider and 59.2% for a local 32B model. We report the EVMBench result as a local extraction rather than an official leaderboard score, and we publish the claim matrix and source artifacts. MIESC is available under AGPL-3.0.

Index Terms*—smart contracts, blockchain security, vulnerability detection, defense-in-depth, multi-agent systems, retrieval-augmented generation, static analysis, formal verification*

I. Introduction

Smart contracts govern over $100 billion in decentralized finance (DeFi) protocols, yet vulnerabilities continue to cause catastrophic losses: the Wormhole bridge exploit ($320M, 2022), the Euler Finance hack ($197M, 2023), and the Curve Finance reentrancy attack ($70M, 2023) underscore the urgency of rigorous security analysis before deployment [1].

Existing automated tools—Slither [2], Mythril [3], Echidna [4]—each implement a single analysis technique and capture only a subset of vulnerabilities. Durieux et al. [5] evaluated 9 tools on 47,587 contracts and found that no single tool achieved both high precision and high recall. In practice, a security researcher preparing a pre-audit report runs 5–10 tools independently, normalizes their heterogeneous outputs by hand, and cross-references findings to filter duplicates—a workflow that takes 4–8 hours per contract and does not scale to the thousands of contracts deployed weekly on Ethereum and its Layer-2 networks.

We address this gap with MIESC, a framework that:

1. Orchestrates **13 external security tools** and **22 internal analysis modules** across **9 complementary defense layers**, each targeting a distinct class of vulnerabilities.
2. Normalizes heterogeneous tool outputs into a unified finding schema with SWC/CWE classification.
3. Filters false positives using a **RAG-enhanced ML pipeline** with 93 versioned vulnerability and security-source documents and per-detector calibration.
4. Maps findings to **12 international security standards** (ISO 27001, NIST CSF, OWASP, CWE, SWC, MITRE ATT&CK).
5. Provides an **extensible plugin system** for community-driven detector development.

6. Implements an **agentic deep-audit loop** that, on CRITICAL findings, queries a primary code-analysis model plus a dedicated verification-role model and applies bounded confidence adjustments (+0.20 on agreement, −0.30 on joint rejection, −0.10 + manual-review flag on disagreement)—turning the LLM into a calibrated verifier rather than a single-shot labeler.

7. Closes the **specification-to-proof loop** by generating Certora CVL rules, Scribble annotations, and SMTChecker assertions directly from findings, and exposing a unified `miesc verify` command that runs the generated specs against the Certora Prover, Halmos, and solc's SMTChecker.

8. Extends coverage to the **Starknet / Cairo ecosystem** with 13 vulnerability classes informed by 2024–2026 real-world exploits (zkLend stale-price, Braavos account re-initialization, Cairo 1.0 unchecked `u256` arithmetic, dropped syscall results, account-abstraction signature replay).

MIESC is designed as a *pre-audit triage tool*, not a replacement for manual expert review. By automating the multi-tool orchestration and normalization workflow, it reduces audit preparation time and enables continuous security monitoring in CI/CD pipelines.

II. Related Work

A. Single-Technique Analysis Tools

Static analysis. Slither [2] uses intermediate representation (SlithIR) to detect vulnerability patterns with low false positive rates. Aderyn [6] implements Rust-based AST analysis for high performance. Solhint provides Solidity-specific linting rules.

Dynamic testing. Echidna [4] performs property-based fuzzing using coverage-guided input generation. Foundry's forge tool combines unit testing with fuzz-based invariant checking. Medusa extends fuzzing with corpus-based strategies.

Symbolic execution. Mythril [3] explores execution paths using SMT solving to find reachable vulnerabilities. Halmos [7] enables symbolic testing within the Foundry framework. Manticore [8] combines symbolic execution with concrete execution.

Formal verification. Certora Prover verifies user-specified properties using SMT-based techniques. SMTChecker is Solidity's built-in formal verification engine.

B. Multi-Tool Frameworks

SmartBugs [9] orchestrates 19 analysis tools with Docker-based execution and SARIF output. It serves primarily as an academic benchmarking framework. SmartBugs 2.0 [10] added parallel execution and improved tool integration.

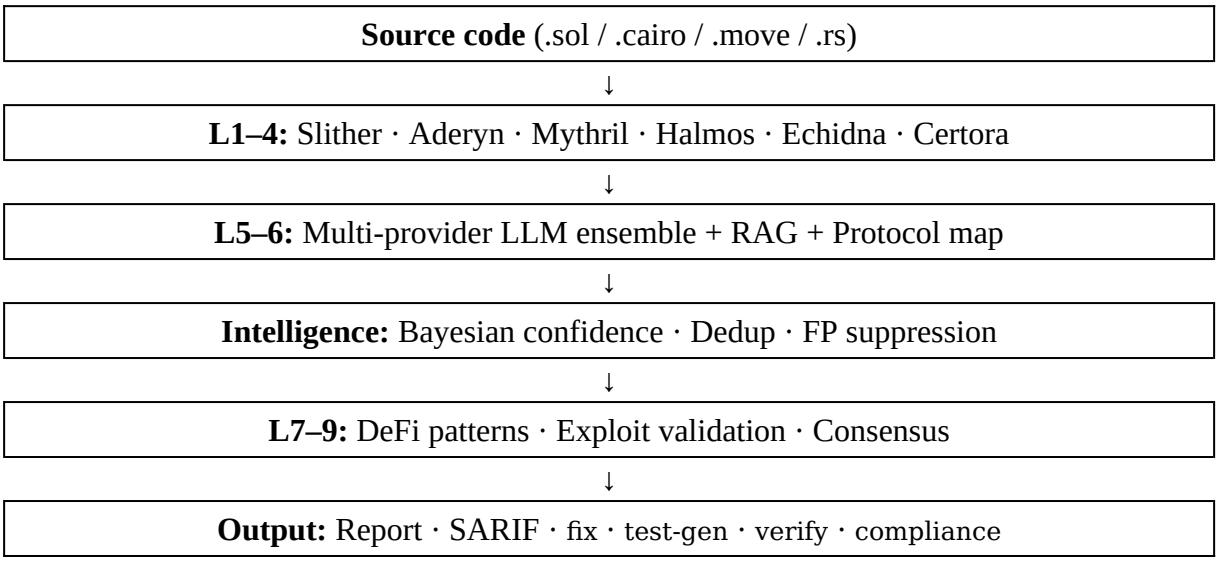
MIESC differs from SmartBugs in three aspects relevant to this evaluation: (1) it integrates 35 analysis modules (13 external + 22 internal) across 9 layers vs. 25 tools, (2) it adds AI/LLM semantic analysis and RAG-enhanced false-positive filtering, and (3) it emits operational artifacts such as SARIF, reports, and GitHub Action outputs for CI/CD integration.

C. AI-Enhanced Security Analysis

GPTScan [11] combines GPT-4 with static analysis for logic vulnerability detection, achieving results that complement pattern-based tools. Other recent approaches apply local LLMs for multi-agent auditing (LLMSmartAudit) and RAG-enhanced code analysis (SmartLLM). LLMBugScanner proposes a multi-LLM ensemble specifically for smart contract bugs. EVMBench [12], curated by OpenAI and Paradigm from 40 Code4rena-derived repositories with 117 curated vulnerabilities, has become a standard benchmark for LLM-based auditing; Cecuro reports 87.7% recall on this benchmark in its public evaluation [13] (a vendor press release rather than a peer-reviewed result). These tools demonstrate the potential of AI-assisted auditing but operate as standalone analyzers. MIESC combines multi-tool static orchestration *with* a multi-provider LLM ensemble, using their outputs alongside static, dynamic, and symbolic results for cross-validated findings.

III. Architecture

MIESC implements a *defense-in-depth* architecture organized into 9 layers, following the principle that multiple independent security mechanisms provide stronger protection than any single mechanism [14]. Figure 1 illustrates the analysis pipeline; each layer targets complementary vulnerability classes (Table 1).



MIESC pipeline: 9 defense layers with multi-provider LLM ensemble and intelligence engine for cross-tool confidence scoring.

MIESC's 9 Defense Layers

Layer	Technique	Ext.	Int.
1	Static Analysis	4	–
2	Dynamic Testing	3	–
3	Symbolic Execution	3	–
4	Formal Verification	3	1
5	AI/LLM Analysis	–	6
6	Pattern Detection	–	5
7	DeFi Security	–	4

8	Exploit Validation	–	3
9	Consensus & Reporting	–	3
	Total	**13**	**22**

Ext. = third-party tools invoked as separate processes. Int. = built-in analysis modules (LLM, pattern matching, consensus).

A. Orchestration Pipeline

The analysis pipeline proceeds as follows:

1. **Input**: Solidity source code (or Vyper, Rust, Move for non-EVM chains).
2. **Tool Execution**: Each layer runs its tools in parallel with configurable timeouts. An adapter pattern normalizes tool-specific outputs into a unified Finding schema.
3. **Aggregation**: The Finding Aggregator deduplicates results using SWC/CWE classification and source location matching.
4. **ML Pipeline**: A false positive filter applies per-detector calibration rates, context analysis (guards, CEI patterns, OpenZeppelin usage), and Solidity version awareness.
5. **RAG Enhancement**: A Retrieval-Augmented Generation system (Section 3.2) enriches findings with vulnerability context from a curated knowledge base.
6. **Report Generation**: Findings are output as JSON, SARIF (for GitHub Security integration), PDF (professional audit reports), or Markdown.

B. RAG System

The RAG component uses ChromaDB as a vector store with sentence embeddings (all-MiniLM-L6-v2, 384 dimensions). It indexes 93 versioned documents across standards, official documentation, benchmark evidence, incident reports, audit guides, tool documentation, and legacy taxonomies. The source-selection policy is documented in the repository and gives highest weight to maintained standards and official documentation, followed by benchmark artifacts, incident analyses, and legacy taxonomies:

- **Reentrancy** (6 patterns): classic, cross-function, read-only, Vyper-specific
- **MEV** (4): sandwich attacks, JIT liquidity, time-bandit
- **Governance** (4): flash loan voting, timelock bypass
- **Proxy** (4): storage collision, uninitialized proxy
- **Bridge/Cross-chain** (3): message replay, oracle manipulation
- **ZK/NFT/DeFi** (6): circuit constraints, royalty bypass, oracle manipulation
- **SWC patterns** (33): covering SWC-100 through SWC-136 (unchecked calls, integer overflow, tx.origin, delegatecall, etc.)

Documents include vulnerability descriptions, severity guidance, exploit references where available (e.g., Curve $70M, Euler $197M, Ronin $624M), structured attack steps, grep-level detection heuristics, and fix templates. The attack steps and detection heuristics are surfaced in the LLM context block, giving the model an actionable procedure rather than a prose description. The knowledge base is versioned (2026-05-06-paper1-source-review-v4); adapters include the RAG version in cache keys so that expanded evidence cannot silently reuse stale LLM responses.

C. False Positive Filter

The FP filter employs a multi-signal approach:

- **Per-detector FP rates**: Empirically calibrated rates for 17+ detectors (e.g., Slither reentrancy: 15%, Mythril integer overflow: 8%).
- **Context analysis**: Detection of reentrancy guards, Checks-Effects-Interactions patterns, and OpenZeppelin/Solmate library usage.
- **Solidity version awareness**: Suppression of integer overflow warnings for Solidity ≥ 0.8.0 (built-in overflow protection).
- **RAG validation**: Cross-reference with curated vulnerability patterns for relevance scoring.

D. Agentic Deep Audit

Beyond batch orchestration, MIESC includes a *DeepAuditAgent* that implements an autonomous 4-phase investigation loop inspired by intelligent agent architectures [15]. Agents communicate via a Model Context Protocol (MCP) bus [16], enabling modular coordination:

1. **Reconnaissance**: Static call graph construction, taint analysis, risk profiling (DeFi, proxy, bridge, token), and attack surface scoring.
2. **Targeted Scan**: Adaptive tool selection based on risk profile—e.g., DeFi contracts trigger oracle and MEV detectors; proxy contracts trigger upgradability checkers.
3. **Deep Investigation**: Iterative agentic loop over HIGH/CRITICAL findings. Each finding triggers *type-specific* follow-up analysis: oracle/price findings invoke a DeFi pattern detector scoped to the vulnerable function, access-control findings generate a Certora CVL rule targeting the function (Section 3.5), and reentrancy/unchecked-call findings are confirmed by targeted taint analysis. RAG enrichment additionally validates whether a known mitigation is present in the code; when present, the finding is downgraded and marked mitigated.
4. **Synthesis**: Cross-finding correlation, exploit chain detection via graph traversal over the call graph, and optional LLM narrative generation (local-first via Ollama, with optional cloud API fallback).

Multi-LLM consensus for critical findings.

For every finding flagged as CRITICAL, Phase 3 runs a two-step LLM verification rather than a single-shot review. MIESC queries (i) the model configured for the code_analysis use case (primary) and (ii) the model configured for the verification use case (verifier). When both models independently confirm the finding, the agent applies a +0.20 confidence boost and

records llm_confirmed. When both reject, the agent applies a −0.30 penalty and downgrades the severity from CRITICAL to HIGH. On disagreement, the agent applies a −0.10 penalty and raises a needs_manual_review flag with the model-level justifications attached, surfacing contested findings for human triage. If the two use cases map to the same model (common in lightweight local deployments), the agent records consensus = single_opinion and leaves confidence unchanged—distinguishing one-model agreement from genuine multi-model agreement. This mechanism is designed to reduce the two failure modes of single-LLM verification: over-confident acceptance of model hallucinations and silent dismissal of real findings.

This agentic approach reduces manual triage by automatically investigating and correlating the most critical findings, detecting multi-step exploit chains that single-pass analysis would miss, and explicitly flagging the subset of findings where automated verdicts disagree.

E. Formal Verification Bridge

A recurring friction point in smart-contract security work is the gap between *finding a bug* and *proving its absence after the fix*. MIESC narrows this gap with a two-command workflow:

- miesc specs consumes a findings JSON and emits executable specifications in three formats: Certora CVL rules such as rule noReentrancy, Scribble inline annotations such as if_succeeds balance decreases, and SMTChecker assertions compatible with solc –model-checker-engine chc.

- miesc verify executes those specifications against whichever prover is installed—the Certora Prover via certoraRun, Halmos via its Foundry integration, or SMTChecker via solc—and normalizes the outputs into a unified VerificationResult (status, rules passed, rules failed, counterexamples, elapsed seconds). A Foundry-root autodetector walks upward from the contract to locate foundry.toml, allowing Halmos to run on projects with standard layouts without additional configuration.

The agentic Phase 3 loop (Section 3.4) closes the inner cycle: when an access-control finding is produced, the agent automatically generates a CVL rule targeting the specific function and attaches it to the finding payload, ready for the operator to pipe into miesc verify –tool certora. This converts an informal finding into an actionable formal obligation in a single step, lowering the activation energy for formal verification in day-to-day audit work.

F. Multi-Chain Support

Although the SmartBugs-curated evaluation focuses on EVM/Solidity, MIESC's architecture treats the analysis chain as a first-class parameter. A miesc analyze dispatcher auto-detects the target language from the file extension and routes to the appropriate chain analyzer:

- **EVM** (.sol, .vy): the 35-module pipeline described above.

- **Starknet / Cairo** (.cairo): a purpose-built analyzer with 13 vulnerability types informed by real 2024–2026 exploits (zkLend $9.6M stale-price read in an empty market; Braavos account re-initialization; Cairo 1.0 unchecked u256 arithmetic; dropped SyscallResult from call_contract_syscall / library_call_syscall; account-abstraction signature replay without nonce binding; and classical felt-overflow / L1 ↔ L2 message / caller spoofing / storage collision / proxy-upgrade / reentrancy / access-control).

- **Move** (.move) and **Solana / Anchor** (.rs): scaffolded analyzers for Sui/Aptos and Solana ecosystems, currently surfaced for community contribution.

The chain-agnostic orchestration layer and the finding-normalization schema are shared across ecosystems, so pattern detection, RAG enrichment, and multi-LLM consensus apply uniformly regardless of target chain.

IV. Evaluation

A. Experimental Setup

We evaluate MIESC on three complementary datasets. First, SmartBugs-curated [5] provides 143 Solidity contracts across 10 vulnerability categories and serves as the classical pattern-based benchmark. Second, an 11-case real-exploit corpus evaluates whether MIESC flags the vulnerability class behind confirmed DeFi incidents. Third, EVMBench [12] evaluates semantic business-logic vulnerabilities from Code4rena-derived audits.

Environment: macOS ARM64 (Apple Silicon), 48GB unified memory. The SmartBugs artifacts reported here were produced with MIESC v5.4.0/v5.4.1 and Python 3.14.4 on 2026-05-06 through 2026-05-09, then consolidated in the Paper 1 v2 evidence freeze on 2026-06-23. EVMBench provider runs use stored MIESC result artifacts; the reproducible ensemble artifact is generated by generate_paper1_artifacts.py, which computes a deterministic union over matched audit:vulnerability keys.

Evaluation profiles: The full-corpus SmartBugs result uses layers 1, 6, and 7 because these layers cover the static analyzers, SmartBugs-specific detectors, ML classifiers, and specialized protocol checks that dominate this benchmark while keeping the run deterministic and fast. To verify that the complete toolchain remains executable, we also run a separate full-tool smoke test over layers 1–9 on the first SmartBugs contract and store its JSON/JSONL artifacts. We treat this smoke run as integration evidence, not as a corpus-level accuracy claim.

B. Results

1) Overall Performance

Results on SmartBugs-curated (143 contracts, 207 ground-truth vulnerabilities)

Approach	Precision	Recall	F1
Slither (alone)*	8.3%	43.2%	13.9%
Mythril (alone)*	6.1%	27.4%	10.0%
Securify*	9.7%	36.8%	15.4%
SmartCheck*	4.2%	18.9%	6.9%
MIESC (latest reproducible local run)	**22.1%**	**95.8%**	**35.9%**

*Baseline from Durieux et al. [5]

Table 2 shows the latest full-corpus reproducible SmartBugs profile stored in the repository as paper1_smartbugs_eval_layers_1_6_7.json. MIESC reaches 95.8% recall and 35.9% F1,

substantially above the single-tool baselines reported by Durieux et al. [5]. This benchmark is useful for classical vulnerability patterns, but it is not the central evidence for business-logic vulnerabilities; that role is played by EVMBench (Section 4.2.6).

The lower precision (9–23%) reflects informational findings (pragma version, naming conventions) that inflate the false positive count. For pre-audit triage, recall is the priority: missed vulnerabilities become production exploits costing millions, while false positives are filtered during manual review at negligible cost.

2) Category-Specific Detection

Performance by Vulnerability Category (SmartBugs-curated)

Category	Precision	Recall	F1
Reentrancy	22.1%	**100%**	36.3%
Unchecked Low-Level Calls	44.8%	**100%**	61.9%
Arithmetic	10.5%	**100%**	19.0%
Time Manipulation	12.5%	**100%**	22.2%
Access Control	27.7%	**100%**	43.4%
Bad Randomness	14.8%	**100%**	25.8%
Denial of Service	11.5%	**100%**	20.7%
Front Running	28.6%	50%	36.4%
Short Addresses	0%	0%	0%
Other	0%	0%	0%

Latest full-corpus local JSON: paper1_smartbugs_eval_layers_1_6_7.json; layers 1, 6, and 7; 143 contracts. Tool-level failures on legacy syntax are counted as per-contract misses when no other layer detects the category.

Seven of ten categories achieve **100% recall**: reentrancy, unchecked low-level calls, arithmetic, time manipulation, access control, bad randomness, and denial of service. Front running achieves 50% recall (2 of 4 contracts); the 2 missed contracts require semantic reasoning about transaction ordering that static patterns cannot capture. The weakest categories are short addresses (a single deprecated-attack-vector contract) and the heterogeneous “other” class (3 contracts with no clear vulnerability pattern). These residual weaknesses motivate the EVMBench analysis, where semantic and protocol-level reasoning are measured directly.

3) Real-World Exploit Validation

To validate beyond academic benchmarks, we evaluated MIESC against 11 confirmed DeFi exploits with $1.59 billion in combined losses, drawn from a larger 25-incident curated corpus (Table 4). For each exploit, the original vulnerable contract source was analyzed and we checked whether MIESC flagged the specific vulnerability class that enabled the exploit.

Detection of Real-World Exploits (11 evaluated contracts, USD 1.59B combined losses)

Vulnerability	Exploits	Detected	Recall
Reentrancy	3	3	**100%**

Access Control	3	3	**100%**
Flash Loan	2	2	**100%**
Arithmetic	1	1	**100%**
Oracle Manipulation	1	0	0%
Governance Attack	1	0	0%
Overall	**11**	**9**	**81.8%**

MIESC achieved 81.8% recall (9/11) on this corpus, with a Cohen's κ of 0.773 (n=11) indicating *substantial agreement* ($\kappa > 0.6$) between MIESC's automated verdicts and the ground truth established by actual on-chain exploits. Given the small sample size, we report this as indicative rather than definitive. Notable detections include the Euler Finance reentrancy ($197M), Ronin Bridge access control ($624M), and Parity wallet freeze ($280M).

4) Execution Performance

Table 5 summarizes MIESC's execution speed for the reproducible SmartBugs full-corpus profile. The run completed all 143 contracts in 737 seconds, or 11.6 contracts per minute.

Execution Performance on SmartBugs-curated

Metric	**Value**
Contracts analyzed	143
Total execution time	737 seconds
Average time per contract	5.15 seconds
Throughput	11.6 contracts/minute
Evaluator completion	100% (143/143 contracts)

5) Full-Tool Integration Smoke

To avoid conflating the fast full-corpus profile with full-tool execution, we also ran a 9-layer smoke test over the first SmartBugs contract. The command used layers 1–9 with a 180s per-tool timeout and produced the full-tool smoke JSON/JSONL artifacts listed in PAPER1_REPRODUCIBILITY.md. It completed in 183.7s with 100.0% recall, 16.7% precision, and 28.6% F1 on that single contract. The JSONL trace records execution across all 9 layers plus the intelligence stage. Unavailable or non-applicable tools are recorded explicitly: for example, Certora requires an external license, Mythril and Manticore require isolated environments, and Vertigo degrades to a skipped result when the input is not a supported Truffle/Hardhat project. This smoke result supports the integration claim; all corpus-level metrics in Tables 2–5 remain tied to the reproducible 143-contract profile.

6) EVMBench: Business Logic Vulnerability Detection

To evaluate MIESC on vulnerabilities requiring semantic understanding—business logic flaws, economic exploits, and protocol-level errors—we benchmark against an EVMBench local high-severity extraction. The official EVMBench paper describes 117 curated vulnerabilities; our local artifact contains 120 high-severity audit findings across 40 audit entries. We report this distinction explicitly and publish the mapping artifact for reproducibility. Unlike SmartBugs' synthetic patterns, EVMBench vulnerabilities require reasoning about protocol invariants, value flows, and multi-contract interactions.

MIESC supports multiple analysis modes through a unified pipeline: the same Chain-of-Thought audit prompt, protocol-specific RAG enrichment, and finding normalization is applied regardless of provider, enabling fair comparison.

Source preparation. For each audit, MIESC selects implementation files by size (excluding tests, mocks, and interfaces), concatenates them into a single context (100KB budget in the recorded EVMBench runs), and prepends protocol context summarizing contract hierarchy, external calls, and extracted invariants. If a provider fails for an audit, the failure is retained in the provider artifact rather than silently removed.

Matching methodology. To determine whether a MIESC finding matches an EVMBench vulnerability, we apply a multi-signal matcher: (1) function name overlap, (2) semantic category matching across 10 vulnerability classes, (3) keyword overlap with stemming, (4) bigram matching, and (5) an LLM judge as a final arbiter when signals 1–4 are inconclusive. This differs from EVMBench's official LLM judge protocol; we publish our matcher and the derived union artifact for reproducibility. Results exhibit variance across runs due to LLM non-determinism, so the paper reports stored run artifacts rather than unstored ad-hoc runs.

EVMBench Recall by Provider and Scale

Mode	**Recall**	**Prec.**	**F1**	**Cost**	*n*
Static only	18.3%	14.5%	16.2%	$0	40
+ Ollama qwen2.5-coder:32b	59.2%	30.1%	39.9%	$0	40
+ GPT-4o	73.7%	20.0%	31.5%	$1.20	39[a]
+ GPT-5	77.5%	42.7%	55.0%	$3	40
+ Claude Sonnet 4.6	82.5%	34.0%	48.2%	$5.50	40
+ Ensemble (all 4)	**92.5%**	—	—	$7	40

[a]GPT-4o artifact contains one clone-failed audit entry; recall is 87/118. Costs are approximate API spend observed for these single stored runs; static and local Ollama require no API calls. Ensemble result is generated by benchmarks/generate_paper1_artifacts.py as the union over audit:vulnerability keys.

Table 6 separates three observations. First, static-only MIESC reaches 18.3% recall on EVMBench, which is useful as a baseline but insufficient for business-logic flaws. Second, a local 32B model (qwen2.5-coder:32b via Ollama) reaches 59.2% recall without API calls, showing that local models are a viable low-cost triage option but not a replacement for stronger reviewers. Third, the best single provider (Claude Sonnet 4.6, 82.5% recall at $5.50/audit) and the four-provider union (**111/120**, **92.5%**) show complementary coverage. The union is numerically above the 87.7% recall publicly reported by Cecuro, but our local extraction and matcher differ from the official EVMBench judge; we therefore treat this as contextual evidence, not a formal leaderboard result.

The union improves recall because providers do not fail on the same findings. Of the 111 detected vulnerabilities, 13 are found by exactly one provider, 16 by two providers, 23 by three providers, and 59 by all four. The 9 vulnerabilities missed by all providers require multi-file reasoning across contract hierarchies that single-prompt analysis does not reliably perform. At approximately $7/audit for the full ensemble (vs. $20K–60K for manual audit engagements), this result is best interpreted as pre-audit triage evidence rather than as a substitute for manual review.

For deployment, the useful distinction is cost and data exposure. A security researcher can run MIESC with a local model and keep the code offline, accepting the lower 59.2% recall. A frontier provider improves recall to 82.5% in the stored run, and the full union improves it further, at higher cost and with provider exposure. Individual LLM runs exhibit variance (about ±4pp on 40 audits in our observations), so all EVMBench results are reported as stored single-run artifacts.

V. Discussion

A. Layer Contribution Analysis

The main pattern is a split between classical vulnerability signatures and semantic reasoning. On SmartBugs-curated, MIESC's static+intelligence profile (layers 1, 6, 7 plus the pattern-based intelligence engine) reaches 95.8% recall (137/143) without invoking any LLM. The 6 remaining misses—3 "other" cases, 2 front-running cases, and 1 short-addresses case—are not uniform failures. The front-running and short-addresses misses are recoverable by a local 32B model, raising the secondary follow-up result to **140/143 (97.9%)**; the 3 "other" contracts remain unresolved because the category does not define a single vulnerability pattern. We therefore use 95.8% as the primary SmartBugs claim and 97.9% as a secondary local-model follow-up.

On EVMBench, static-only analysis reaches 18.3% recall, while LLM providers detect complementary subsets of business-logic vulnerabilities and the four-provider union closes 12 otherwise missed findings beyond the best single provider (Claude Sonnet 4.6, 99/120). The evidence supports a narrower claim than "more tools is better": each layer is useful only when matched to the vulnerability class it can actually observe.

B. Comparison with SmartBugs 2.0

SmartBugs 2.0 [10] is the closest comparable framework, integrating 25 analysis tools with Docker-based execution. However, SmartBugs focuses on providing a *benchmarking platform* and does not publish detection metrics (precision, recall) on its own curated dataset. The tools it integrates (Mythril, Slither, Securify, Oyente, etc.) are individually benchmarked in Durieux et al. [5], where the best single tool achieves 43.2% recall (Slither).

A direct recall comparison is not possible because SmartBugs does not publish framework-level detection metrics on the curated dataset. However, the individual tools it integrates achieve at most 43.2% recall independently [5]. MIESC's latest full-corpus SmartBugs profile reaches 95.8% recall, and its EVMBench ensemble reaches 92.5% recall on the local high-severity extraction. The differentiator is not tool count alone but (1) normalization across heterogeneous tool outputs, (2) cross-tool confidence and false-positive filtering, and (3) LLM layers that detect vulnerability classes—front running, bad randomness, and business logic—that static analysis structurally struggles to reach.

C. False Positive Management

Precision (22.1%) remains a usability concern. The latest SmartBugs-curated run reports 481 false positives for 143 contracts, dominated by broad detector categories and informational issues (pragma version, naming conventions). The FP filter addresses this through four mechanisms: (1) per-detector calibration based on empirical FP rates from our benchmark runs (e.g., Slither's `solc-version` detector has near-100% FP rate on modern contracts), (2) Solidity version awareness that suppresses integer overflow warnings for contracts using ≥ 0.8 (which has built-in overflow protection), (3) library detection that recognizes OpenZeppelin and Solmate safe patterns, and (4) context-aware filters that suppress patterns in benign contexts (e.g., `block.timestamp` used only in timelock `require()`, block variables not used for entropy generation, pre-0.8 contracts without actual financial arithmetic). After filtering, the actionable finding count drops to 2–3 per contract on average. Since v5.2.0, the intelligence engine performs semantic deduplication, context-aware FP suppression, and cross-tool Bayesian confidence scoring. A trainable FP classifier (GradientBoosting) is included but awaits a larger labeled auditor corpus for production deployment.

D. Analysis of Missed Exploits

Two real-world exploits were not detected: the BonqDAO oracle manipulation ($120M) and the Beanstalk governance flash loan attack ($182M). Both require *semantic understanding* of protocol-specific logic that static pattern matching cannot capture. BonqDAO's vulnerability was in the interaction between a Tellor oracle and a custom stablecoin—detecting this requires modeling the oracle's trust assumptions, not just code patterns. Beanstalk's attack exploited governance voting with flash-borrowed tokens in a single transaction—a logical flaw invisible to pattern-based analysis. These findings motivate future work on protocol-aware semantic analysis in Layers 5–7.

E. Threats to Validity

Internal validity. The SmartBugs-curated ground truth was established by Durieux et al. [5] and may contain labeling errors. Our finding classifier maps MIESC tool outputs to SmartBugs categories using SWC IDs and keyword matching, which may introduce classification errors. EVMBench matching also differs from the official judge protocol: our local extraction uses a multi-signal matcher and stored provider artifacts. We mitigate this by publishing the matcher, provider JSONs, and the deterministic ensemble artifact.

External validity. The SmartBugs benchmark contains contracts from 2016–2020, which may not represent modern DeFi protocols. We address this with the real-world exploit evaluation (2020–2023 exploits), though the 11-exploit sample is small, and with EVMBench business-logic findings from Code4rena-derived audits. The official EVMBench paper reports 117 curated vulnerabilities, while our local artifact evaluates 120 high-severity findings; this difference is explicitly documented in the reproducibility files.

Construct validity. Recall is our primary metric because missed vulnerabilities have higher cost than false positives in pre-audit triage. However, precision remains important for usability—a tool that generates too many false positives will be abandoned by practitioners.

Reproducibility. All benchmark scripts, ground truth data, and evaluation methodology are included in the repository. The EVMBench ensemble and claims matrix can be reproduced with:

```
SOURCE_DATE_EPOCH=0 python3 benchmarks/generate_paper1_artifacts.py
```

This generates evmbench_ensemble_40.json and paper1_claims_matrix.json. SmartBugs can be reproduced with the command listed in PAPER1_REPRODUCIBILITY.md; the fixed full-corpus profile is miesc evaluate corpus ... --layers 1,6,7 --timeout 120, and the local-Ollama lift is recorded in the v2 SmartBugs follow-up JSON named there. The same document records the full-tool smoke command for layers 1–9. The Slither adapter selects an installed solc-select artifact compatible with each contract pragma (for example, ^0.4.x contracts use solc-0.4.26), avoiding the historical failure mode where solc 0.8.20 was applied to legacy Solidity.

F. Limitations

- The Starknet/Cairo analyzer originally used line-by-line regex matching; as of v5.1.9, a block-level scanner (re.DOTALL with a line-offset table for accurate source mapping) handles multi-line patterns such as #[external] followed by fn foo(...), closing the previously reported gap.
- Move (Sui/Aptos) and Solana/Anchor analyzers are scaffolded for community contribution but not yet evaluated against ground-truth benchmarks.
- Full 9-layer analysis is substantially slower than the reproducible full-corpus profile because it invokes LLM and formal-verification adapters; in the recorded smoke run, one contract required 183.7s.
- Tool availability varies by platform (Echidna/Medusa are amd64-only).
- LLM-based layers depend on model quality and introduce non-determinism; the multi-LLM consensus mechanism (Section 3.4) mitigates but does not eliminate this; results may vary across Ollama model versions.
- The trainable AuditorTrainedFPClassifier shipped with the framework is deployed but dormant: activating it requires a labeled corpus of auditor-validated findings, which we are collecting as an ongoing effort.
- The real-world exploit evaluation (11 contracts) is limited in size; expanding to 100+ exploits and adding a Starknet-specific exploit corpus (zkLend, Braavos, Nostra) is planned.

VI. Conclusion

This work reports a staged evaluation of MIESC rather than a single headline score. On pattern-based vulnerabilities (SmartBugs), the static+intelligence profile reaches 95.8% recall (137/143) without any LLM; the local-model follow-up raises the secondary result to **97.9%** (140/143). On business-logic vulnerabilities (EVMBench local high-severity extraction), static-only MIESC detects 22/120 findings (18.3%), a local 32B model reaches 59.2%, and the four-provider union reaches **111/120 detections** (**92.5%** recall). The EVMBench comparison to Cecuro is informative but not official-leaderboard equivalent because the local extraction and matcher differ. The provider-diversity artifact explains why the union improves recall: 13 vulnerabilities are detected by exactly one provider, 16 by two, 23 by three, and 59 by all four.

The practical contribution is the workflow boundary. miesc scan contract.sol --model ollama runs Slither, Aderyn, an intelligence engine with Bayesian cross-tool confidence, protocol-specific RAG enrichment, and a local LLM audit without sending code to a cloud provider. Teams can

then decide whether the remaining findings justify a frontier provider or manual review. A single frontier provider (--model claude, $5.50/audit in the stored run) reaches 82.5% recall; the full ensemble ($7/audit) reaches 92.5%.

The agentic and formal-verification components follow the same principle: an isolated LLM verdict or a standalone prover run is rarely sufficient, but each can add evidence when its assumptions are explicit. MIESC therefore records LLM consensus, generated specifications, and prover inputs as separate artifacts rather than collapsing them into a single confidence label. The 2024–2026 Cairo exploit coverage applies this idea to a second ecosystem by encoding lessons from zkLend and Braavos as machine-checkable patterns.

Three directions for future work emerge from these results. First, precision needs improvement—the current 2–3 actionable findings per contract after filtering is usable, but the trainable FP classifier included in MIESC is waiting on a labeled auditor corpus to reduce this further. Second, the 11-exploit evaluation should expand to 100+ contracts (including a Starknet/Cairo exploit corpus) to strengthen statistical claims. Third, MIESC v5.3.0 introduced `miesc fix` for automated text-level remediation—inserting `nonReentrant` modifiers, access-control guards, and `SafeERC20` wrapping; detailed remediation metrics (apply, compile, and vulnerability-elimination rates) are reported separately in the companion remediation study, alongside `miesc compliance` for mapping findings to 12 international standards. Generating Foundry test harnesses from exploit scenarios to formally verify that each fix discharges the original CVL rule remains future work.

MIESC is available under AGPL-3.0 (see *Data Availability*).

Acknowledgment

Acknowledgments are omitted in this version for double-blind review. The authors thank the Ethereum security community and the developers of Slither, Mythril, Echidna, Foundry, Aderyn, and Halmos for their foundational contributions.

Data Availability

MIESC source code, benchmark scripts, ground truth datasets, and evaluation results are publicly available at the project repository (link withheld for double-blind review) under AGPL-3.0. The SmartBugs-curated dataset is from Durieux et al. [5]. The real-world exploit dataset was curated from DeFiHackLabs (https://github.com/SunWeb3Sec/DeFiHackLabs). Paper-specific reproducibility artifacts are documented in `PAPER1_REPRODUCIBILITY.md`, with generated outputs `paper1_claims_matrix.json`, `evmbench_static_40.json`, `evmbench_ensemble_40.json`, and `paper1_smartbugs_local_ollama_followup_20260623.json`.

References

1. N. Atzei, M. Bartoletti, and T. Cimoli, “A survey of attacks on Ethereum smart contracts (SoK),” *Proceedings of the 6th International Conference on Principles of Security and Trust (POST)*, pp. 164–186, 2017.

2. J. Feist, G. Grieco, and A. Groce, “Slither: A static analysis framework for smart contracts,” in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.

3. B. Mueller, “Smashing Ethereum smart contracts for fun and real profit,” in *HITB Security Conference*, 2018.

4. G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, “Echidna: Effective, usable, and fast fuzzing for smart contracts,” in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.

5. T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, “Empirical review of automated analysis tools on 47,587 Ethereum smart contracts,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. ACM, 2020, pp. 530–541.

6. Cyfrin, “Aderyn: Rust-based Solidity ast analyzer,” https://github.com/Cyfrin/aderyn, 2024.

7. a16z crypto, “Halmos: Symbolic testing for EVM smart contracts,” https://github.com/a16z/halmos, 2023.

8. M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, “Manticore: A user-friendly symbolic execution framework for binaries and smart contracts,” in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 1186–1189.

9. J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, “SmartBugs: A framework to analyze Solidity smart contracts,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 1349–1352.

10. M. di Angelo, T. Durieux, J. F. Ferreira, and G. Salzer, “SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts,” in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2102–2105.

11. Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, “GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.

12. A. Gupta, V. Gottimukkala, D. Robinson, Paradigm, and OpenAI, “EVMBench: Evaluating AI agents on smart contract security,” *arXiv preprint arXiv:2603.04915*, 2026. [Online]. Available: https://github.com/paradigmxyz/evmbench

13. Cecuro, “AI audit firm cecuro outperforms nearest rival by 2x on OpenAI smart contract exploit benchmark,” https://chainwire.org/2026/04/16/ai-audit-firm-cecuro-outperforms-nearest-rival-by-2x-on-openai-smart-contract-exploit-benchmark/, 2026, press release; vendor-reported result, not a peer-reviewed evaluation.

14. J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

15. M. Wooldridge and N. R. Jennings, “Intelligent agents: Theory and practice,” *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.

16. Anthropic, “Model context protocol specification,” https://modelcontextprotocol.io, 2024.